SR - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continous color palettes
```

```
# load needed functions
# source(paste0(RootDir,"/src/function_library/sp_functions.R"))
# do this in the calling R script
```

```
# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat," - ",CurTask," - ",RunLabel," - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE
```

```
CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres
```

```
# done in calling script
# print(paste0("Using root dir: ",RootDir))
# RunDir <- paste0(paste0(RootDir,"/output/",RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir,showWarnings = TRUE)
#   print(paste0("created run directory: ",RunDir))
# }
```

```
# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir,showWarnings = TRUE)
#   print(paste0("created participant directory: ",PatientDir))
# }
# TablesDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir,showWarnings = TRUE)
#   print(paste0("created tables directory: ",TablesDir))
# }
# FigDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir,showWarnings = TRUE)
#   print(paste0("created figures directory: ",FigDir))
# }
# ReportsDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir,showWarnings = TRUE)
#   print(paste0("created reports directory: ",ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())
```

```
ModelDatFilename<-paste0(RootDir,"/data/",CurPat,"/",CurPat,"_",CurTask,"_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ",ModelDatFilename))
  print(sprintf("\troot dir: ",RootDir))
}
PosDat<-read.csv(ModelDatFilename)
```

```
# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values",OrigDataLen))
```

```
## [1] "Original data has 4416 values"
```

```
if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values",DataLen))
  print(sprintf("%d values eliminated.",OrigDataLen - DataLen))
}
```

```
## [1] "Data without fractional preserved values has 4375 values"
## [1] "41 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite

# get counts of syllable components in different serial positions
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syl

syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))

kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	556	35	127	NA	NA	718
2	65	NA	441	94	108	708
3	310	NA	169	213	16	708
4	308	NA	239	68	38	653
5	234	NA	213	71	39	557
6	208	1	138	73	22	442
7	181	NA	102	28	19	330
8	91	NA	56	24	4	175
9	75	NA	2	NA	7	84

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7743733	0.0487465	0.1768802	NA	NA	718
2	0.0918079	NA	0.6228814	0.1327684	0.1525424	708
3	0.4378531	NA	0.2387006	0.3008475	0.0225989	708
4	0.4716692	NA	0.3660031	0.1041348	0.0581930	653
5	0.4201077	NA	0.3824057	0.1274686	0.0700180	557
6	0.4705882	0.0022624	0.3122172	0.1651584	0.0497738	442
7	0.5484848	NA	0.3090909	0.0848485	0.0575758	330
8	0.5200000	NA	0.3200000	0.1371429	0.0228571	175
9	0.8928571	NA	0.0238095	NA	0.0833333	84

```
# get percentages only from summary table
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
syll_comp_perc <- syll_comp_perc %>% rename("Coda" ="1","Satellite"="S")

# pivot to long format for plotting
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
            names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda","Sa
```

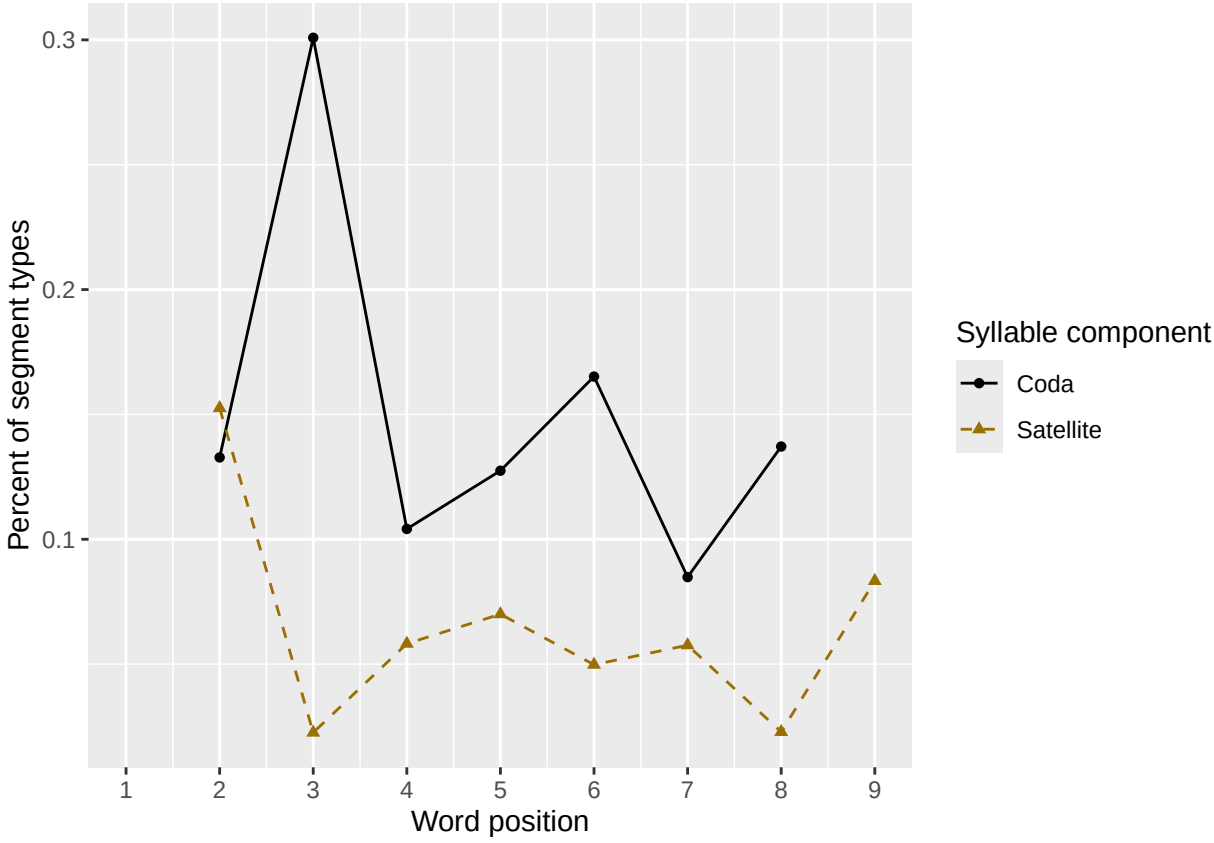
```
# plot satellite and coda occurances across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                         aes(x=pos,y=percent,group=syll_component,
                             color=syll_component,
                             linetype = syll_component,
                             shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_pos_of_syll_complexities_in_stimuli.png"),plot=sy
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`)
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```




```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```
min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,"_preserved_percentages_by_len_
pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.95  1     0.983 NA     NA     NA     NA     NA     NA
## 2       5 0.907 0.959 0.959  0.948 NA     NA     NA     NA     NA
## 3       6 0.950 0.983 0.983  0.975  0.975 NA     NA     NA     NA
## 4       7 0.832 0.910 0.955  0.921  0.946  0.929 NA     NA     NA
## 5       8 0.903 0.974 0.947  0.948  0.942  0.961  0.974 NA     NA
## 6       9 0.867 0.933 0.920  0.944  0.879  0.944  0.967  0.967 NA
## 7      10 0.964 0.95  0.877  0.938  0.901  0.917  0.952  0.976  0.952
```

```
# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved),N=n()) %>% dply
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                 "_N_by_len_position.csv"))
pos_len_N_table
```

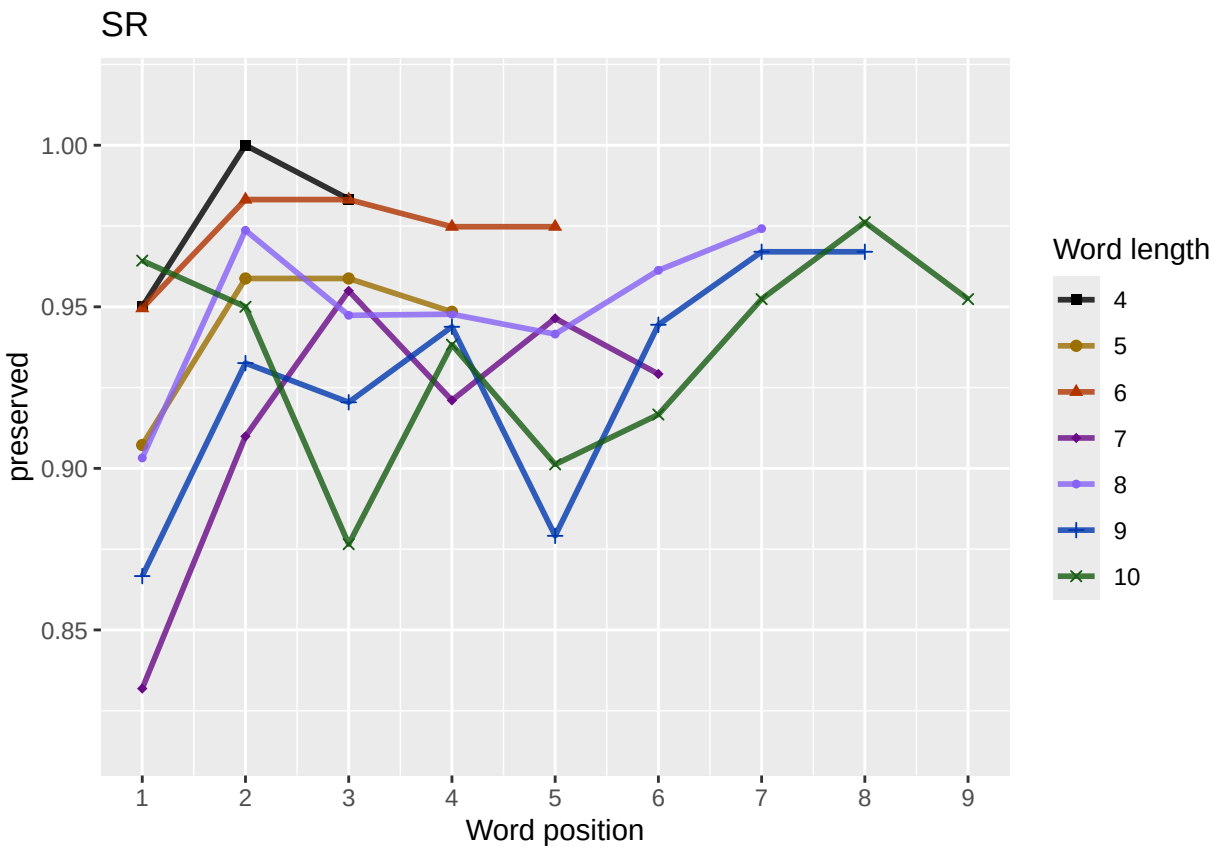
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##     <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1       4    60    60    60    NA    NA    NA    NA    NA    NA
## 2       5    97    97    97    97    NA    NA    NA    NA    NA
## 3       6   119   119   119   119   119    NA    NA    NA    NA
## 4       7   113   111   111   114   112   113    NA    NA    NA
## 5       8   155   152   152   153   154   155   155    NA    NA
## 6       9    90    89    88    89    91    90    91    91    NA
## 7      10    84    80    81    81    81    84    84    84    84
```

```
obs_linetypes <- c("solid","solid","solid","solid",
                   "solid","solid","solid","solid","solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen<-factor(pos_len_summary$stimlen)
pos_len_summary$pos<-factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimle
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                           paste0(CurPat),
                                           NULL,
                                           c(min_preserved,max_preserved),
                                           palette_values,
                                           shape_values,
                                           obs_linetypes,
                                           pred_linetypes)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot,device="png",unit="cm",width=15,height=11)
len_pos_plot
```



Length and position

```
# length and position

LPModelEquations<-c("preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

LPRes<-TestModels(LPModelEquations,PosDat)
```

```
## ****************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          0.94661           0.15612          -0.22684           1.91088           0.02613          -0
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4369 Residual
## Null Deviance:       1928
## Residual Deviance: 1898  AIC: 1910
## log likelihood:  -949.0542
## Nagelkerke R2:  0.01893564
## % pres/err predicted correctly:  -470.7198
## % of predictable range [ (model-null)/(1-null) ]:  0.00684216
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      3.5158      -0.1549       0.1315
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1905  AIC: 1911
## log likelihood:  -952.5719
## Nagelkerke R2:  0.01445004
## % pres/err predicted correctly:  -471.5453
## % of predictable range [ (model-null)/(1-null) ]:  0.005104156
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     2.97185     -0.08989      0.32675     -0.02268
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1904  AIC: 1912
## log likelihood:  -952.0387
## Nagelkerke R2:  0.01513042
## % pres/err predicted correctly:  -471.3912
## % of predictable range [ (model-null)/(1-null) ]:  0.005428453
## ***************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
```

```
##    3.471371    -0.153614    -0.002887     0.155073
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1905  AIC: 1913
## log likelihood:  -952.5511
## Nagelkerke R2:  0.01447656
## % pres/err predicted correctly:  -471.5334
## % of predictable range [ (model-null)/(1-null) ]:  0.005129154
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     2.46532      0.09017
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1919  AIC: 1923
## log likelihood:  -959.6872
## Nagelkerke R2:  0.005354825
## % pres/err predicted correctly:  -473.056
## % of predictable range [ (model-null)/(1-null) ]:  0.001923393
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.32851     -0.01111      0.18184
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1919  AIC: 1925
## log likelihood:  -959.3746
## Nagelkerke R2:  0.00575507
## % pres/err predicted correctly:  -472.9335
## % of predictable range [ (model-null)/(1-null) ]:  0.002181481
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.5430      -0.0963
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
```

```
## Null Deviance:       1928
## Residual Deviance: 1922  AIC: 1926
## log likelihood:  -960.8212
## Nagelkerke R2:  0.00390256
## % pres/err predicted correctly:  -473.3422
## % of predictable range [ (model-null)/(1-null) ]:  0.001320861
## ****************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.795
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4374 Residual
## Null Deviance:       1928
## Residual Deviance: 1928  AIC: 1930
## log likelihood:  -963.8655
## Nagelkerke R2:  -6.230811e-16
## % pres/err predicted correctly:  -473.9696
## % of predictable range [ (model-null)/(1-null) ]:  0
## ****************************
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                         AIC=LPRes$AIC,
                         row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)
kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	1910.108	0.000000	1.000000	0.4554096	0.0189356	2.946608	0.1561168	9.108750	- 0.2097085	- 0.2268413	0.0261339
preserved ~ stimlen + pos	1911.144	1.035342	0.5959069	0.2713810	0.0144563	2.515825	8 - 0.1548783	0.1314884	NA	NA	NA
preserved ~ stimlen * pos	1912.077	1.968938	0.3736376	0.1701581	0.0151324	2.971846	5 - 0.0898874	0.3267481	- 0.0226805	NA	NA
preserved ~ stimlen + I(pos^2) + pos	1913.102	2.993785	0.2238246	0.1019310	0.0144766	2.647137	12 - 0.1536145	0.1550730	NA	- 0.0028866	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ pos	1923.374	13.265902	0.0013163	0.0005599	0.4005354	1.284653217	NA	0.0901687	NA	NA	NA
preserved ~ I(pos^2) + pos	1924.749	14.640665	0.0006610	0.0003010	0.3957552	1.213285119	NA	0.1818352	NA	-0.0111058	NA
preserved ~ stimlen	1925.642	15.533879	0.0004235	0.0001929	0.3903926	1.265429662	-0.0963028	NA	NA	NA	NA
preserved ~ 1	1929.731	19.622591	0.0000548	0.0000250	0.0000000	1.207949072	NA	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen * (I(pos^2) + pos)"
```

```
print(BestLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stimle
##          0.94661           0.15612          -0.22684           1.91088           0.02613          -0
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4369 Residual
## Null Deviance:       1928
## Residual Deviance: 1898  AIC: 1910
```

```
PosDat$LPFitted<-fitted(BestLPModel)
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
                        NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

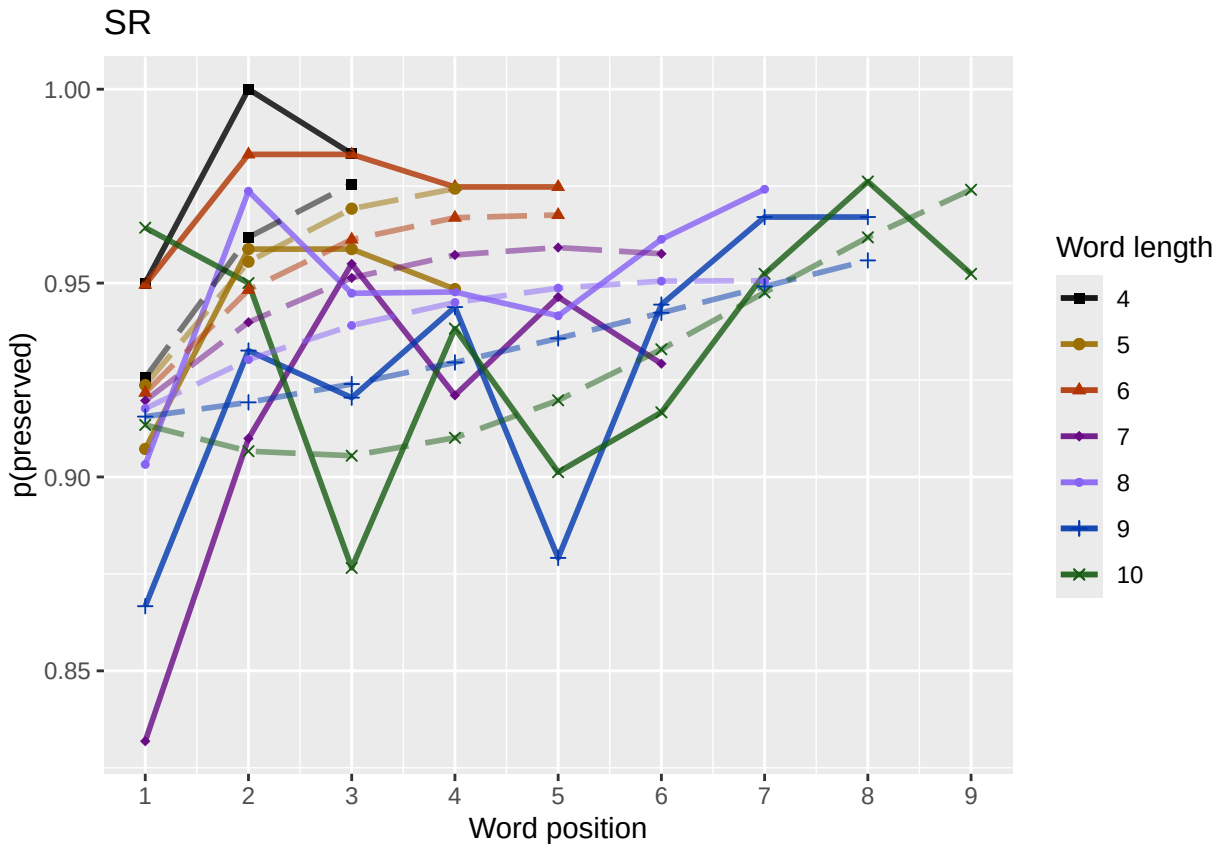
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##     <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1       4 0.926 0.962 0.976 NA     NA     NA     NA     NA     NA
## 2       5 0.924 0.956 0.969  0.974 NA     NA     NA     NA     NA
## 3       6 0.922 0.948 0.961  0.967  0.968 NA     NA     NA     NA
## 4       7 0.920 0.940 0.951  0.957  0.959  0.958 NA     NA     NA
## 5       8 0.918 0.930 0.939  0.945  0.949  0.951  0.951 NA     NA
## 6       9 0.916 0.919 0.924  0.930  0.936  0.942  0.949  0.956 NA
## 7      10 0.913 0.907 0.905  0.910  0.920  0.933  0.948  0.962  0.974
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
```

```
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                          paste0(PosDat$patient[1]),
                                          "LPFitted",
                                          NULL,
                                          palette_values,
                                          shape_values,
                                          obs_linetypes,
                                          pred_linetypes = c("longdash")
                                          )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_percent_preserved_by_length_pos_wfit.png"),plot=
fitted_len_pos_plot
```



length and position without fragments to see if this changes position^2 influence

```
# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
```

```
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array
```

```
# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag
```

```
## # A tibble: 1 x 2
##   frag_sum     N
##      <int> <int>
## 1       11   720
```

```
num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100
```

```
## Warning: Unknown or uninitialised column: `percent_with_frag`.
```

```
print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
              num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))
```

```
## [1] "The number of responses with fragments was 11 / 720 = 1.53 percent"
```

```
write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_percent_fragments.csv"),row.names = FALSE)
```

```
# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)
```

```
## ****************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          0.79426           0.18006          -0.20634           1.93870           0.02577          -0
##
## Degrees of Freedom: 4338 Total (i.e. Null);  4333 Residual
## Null Deviance:       1717
## Residual Deviance: 1670  AIC: 1682
## log likelihood:  -835.2195
## Nagelkerke R2:  0.03273982
## % pres/err predicted correctly:  -405.1146
## % of predictable range [ (model-null)/(1-null) ]:  0.01066992
## ****************************
## model index:  4
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      3.3254      -0.1483       0.2210
##
## Degrees of Freedom: 4338 Total (i.e. Null);  4336 Residual
## Null Deviance:       1717
## Residual Deviance: 1677  AIC: 1683
## log likelihood:  -838.6858
## Nagelkerke R2:  0.02789959
## % pres/err predicted correctly:  -405.78
## % of predictable range [ (model-null)/(1-null) ]:  0.009049054
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     2.64680     -0.06658      0.48545     -0.03101
##
## Degrees of Freedom: 4338 Total (i.e. Null);  4335 Residual
## Null Deviance:       1717
## Residual Deviance: 1676  AIC: 1684
## log likelihood:  -837.8989
## Nagelkerke R2:  0.02899904
## % pres/err predicted correctly:  -405.5551
## % of predictable range [ (model-null)/(1-null) ]:  0.009596991
## ***************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##    3.400599    -0.150285     0.005619     0.177522
##
## Degrees of Freedom: 4338 Total (i.e. Null);  4335 Residual
## Null Deviance:       1717
## Residual Deviance: 1677  AIC: 1685
## log likelihood:  -838.6299
## Nagelkerke R2:  0.02797764
## % pres/err predicted correctly:  -405.8152
## % of predictable range [ (model-null)/(1-null) ]:  0.008963241
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
## (Intercept)          pos
##      2.3103       0.1846
##
## Degrees of Freedom: 4338 Total (i.e. Null);  4337 Residual
## Null Deviance:       1717
## Residual Deviance: 1689  AIC: 1693
## log likelihood:  -844.6232
## Nagelkerke R2:  0.01959057
## % pres/err predicted correctly:  -406.9002
## % of predictable range [ (model-null)/(1-null) ]:  0.00632023
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##    2.280178    -0.002774     0.206213
##
## Degrees of Freedom: 4338 Total (i.e. Null);  4336 Residual
## Null Deviance:       1717
## Residual Deviance: 1689  AIC: 1695
## log likelihood:  -844.6095
## Nagelkerke R2:  0.01960986
## % pres/err predicted correctly:  -406.8706
## % of predictable range [ (model-null)/(1-null) ]:  0.006392271
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##     3.41022     -0.05976
##
## Degrees of Freedom: 4338 Total (i.e. Null);  4337 Residual
## Null Deviance:       1717
## Residual Deviance: 1715  AIC: 1719
## log likelihood:  -857.5273
## Nagelkerke R2:  0.001453677
## % pres/err predicted correctly:  -409.309
## % of predictable range [ (model-null)/(1-null) ]:  0.0004520197
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
```

```
##       2.949
##
## Degrees of Freedom: 4338 Total (i.e. Null);  4338 Residual
## Null Deviance:       1717
## Residual Deviance: 1717  AIC: 1719
## log likelihood:  -858.5582
## Nagelkerke R2:  3.397064e-16
## % pres/err predicted correctly:  -409.4946
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                        AIC=NoFrag_LPRes$AIC,
                        row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	1682.439	0.000000	1.000000	0.4193702	0.2032739	0.8794257	0.1800582	0.9387033	-0.2182568	-0.2063441	0.0257688
preserved ~ stimlen + pos	1683.372	0.932457	0.627368	0.2630977	0.1702789	0.6325434	0.1483108	0.2209842	NA	NA	NA
preserved ~ stimlen * pos	1683.798	1.358742	0.506935	0.2125987	0.1702899	0.6467973	0.0665763	-0.4854490	0.0310083	NA	NA
preserved ~ stimlen + I(pos^2) + pos	1685.262	2.820765	0.244049	0.1023477	0.1027977	0.4005995	0.1502854	-0.1775222	NA	0.0056186	NA
preserved ~ pos	1693.246	10.807348	0.004500	0.0001887	0.0195926	0.3103046	NA	0.1846207	NA	NA	NA
preserved ~ I(pos^2) + pos	1695.219	12.779810	0.001678	0.0000703	0.0196092	0.2801784	NA	0.2062134	NA	-0.0027739	NA
preserved ~ stimlen	1719.053	36.615471	0.000000	0.0000000	0.0001453	0.0741021	50.0597593	NA	NA	NA	NA
preserved ~ 1	1719.113	36.677369	0.000000	0.0000000	0.0000000	0.0720490	579 NA	NA	NA	NA	NA

```
# plot no fragment data

NoFragData$LPFitted<-fitted(NoFragBestLPModel)
```

```
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
                          NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

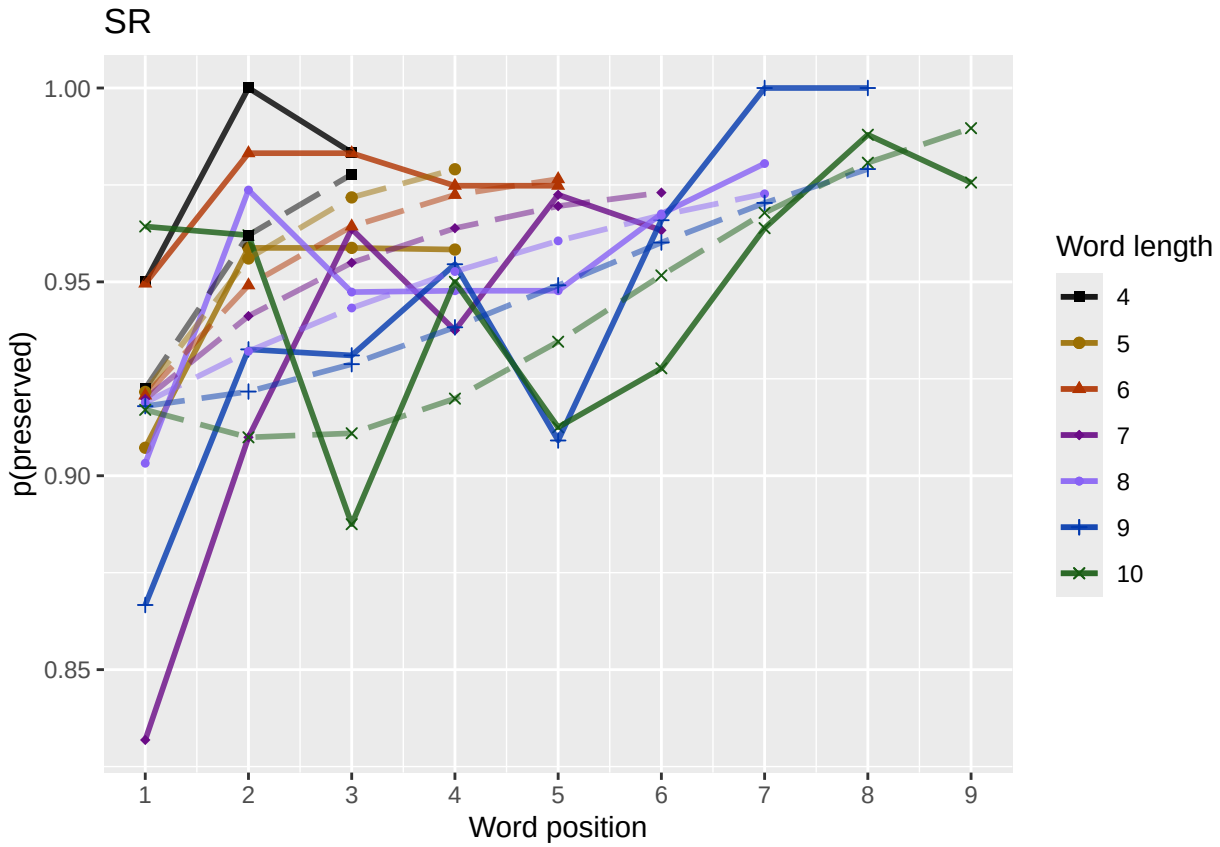
```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.923 0.962 0.978 NA     NA     NA     NA     NA     NA
## 2       5 0.922 0.956 0.972  0.979 NA     NA     NA     NA     NA
## 3       6 0.921 0.949 0.964  0.972  0.977 NA     NA     NA     NA
## 4       7 0.920 0.941 0.955  0.964  0.970  0.973 NA     NA     NA
## 5       8 0.919 0.932 0.943  0.953  0.961  0.967  0.973 NA     NA
## 6       9 0.918 0.922 0.929  0.938  0.949  0.960  0.970  0.979 NA
## 7      10 0.917 0.910 0.911  0.920  0.935  0.952  0.968  0.981  0.990
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
                                        paste0(NoFragData$patient[1]),
                                        "LPFitted",
                                        NULL,
                                        palette_values,
                                        shape_values,
                                        obs_linetypes,
                                        pred_linetypes = c("longdash")
                                        )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "no_fragments_percent_preserved_by_length_pos_wfit.png"),
       plot=nofrag_fitted_len_pos_plot,
       device="png",unit="cm",
       width=15,height=11)
nofrag_fitted_len_pos_plot
```



back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.82 - 1.02"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account

# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]

# take averages for positions
# don't want downward estimates influenced by return upward of U
# therefore, for downward influence, use only the values before the min
# take the difference between each value (differences between position proportion correct) **NOTE** pro
# average the difference in probabilities

# in case min is close to the end or we are not using a min (for non-U-shaped)
# functions, we need to get differences _first_ and then average those (e.g.
# if there is an effect of length and we just average position data,
# later positions) will have a lower average (because of the length effect)
# even if all position functions go upward
```

```
table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional length: "
```

```
print(OA_mean_len_diff)
```

```
## [1] -0.007717739
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding  U-shape): "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional position (excluding  U-shape): "
```

```
print(OA_mean_pos_diff)
```

```
## [1] 0.009970405
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)
```

```
if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                               2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
```

```
    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}
```

```
## [1] "No U-shape in this participant"
```

```
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upwa
```

```
  CurrentLabel<-"return upward value"
  print(upward_dist[biggest_return_upward_row])
   results_report_DF <- AddReportLine(results_report_DF,
                                      CurrentLabel,
                                      upward_dist[biggest_return_upward_row])

  print(" ")
  # percentage return upward
  percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upwar
  CurrentLabel <- "Return upward as a proportion of the downward distance:"
  print(CurrentLabel)
  print(percentage_return_upward)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                         percentage_return_upward)
}else{
  print("no U-shape in this participant")
}
```

```
## [1] "no U-shape in this participant"
```

```
FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
            "preserved ~ stimlen*log_freq",
            "preserved ~ stimlen+log_freq",
            "preserved ~ pos*log_freq",
            "preserved ~ pos+log_freq",
            "preserved ~ stimlen*log_freq + pos*log_freq",
            "preserved ~ stimlen*log_freq + pos",
            "preserved ~ stimlen + pos*log_freq",
            "preserved ~ stimlen + pos + log_freq",
            "preserved ~ (I(pos^2)+pos)*log_freq",
            "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen*log_freq + I(pos^2) + pos",
            "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen + I(pos^2) + pos + log_freq",

            # models without frequency
            "preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)
```

```
## ***************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
```

```
## Coefficients:
## (Intercept)      stimlen          pos     log_freq
##     3.32006     -0.12818      0.13188      0.08446
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1899  AIC: 1907
## log likelihood:  -949.7366
## Nagelkerke R2:  0.01806604
## % pres/err predicted correctly:  -470.9142
## % of predictable range [ (model-null)/(1-null) ]:  0.006432849
## **************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)       stimlen           pos      log_freq  pos:log_freq
##      3.32152      -0.13112       0.13994       0.03504       0.01415
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4370 Residual
## Null Deviance:       1928
## Residual Deviance: 1899  AIC: 1909
## log likelihood:  -949.3671
## Nagelkerke R2:  0.01853695
## % pres/err predicted correctly:  -470.8746
## % of predictable range [ (model-null)/(1-null) ]:  0.006516118
## **************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq
##          3.30602          -0.12786           0.20263           0.13182          -0.01491
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4370 Residual
## Null Deviance:       1928
## Residual Deviance: 1899  AIC: 1909
## log likelihood:  -949.4879
## Nagelkerke R2:  0.01838303
## % pres/err predicted correctly:  -470.8524
## % of predictable range [ (model-null)/(1-null) ]:  0.006563007
## **************************
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos     log_freq
##    3.273192    -0.126845    -0.003043     0.156735     0.084491
```

```
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4370 Residual
## Null Deviance:       1928
## Residual Deviance: 1899  AIC: 1909
## log likelihood:  -949.7136
## Nagelkerke R2:  0.01809535
## % pres/err predicted correctly:  -470.9006
## % of predictable range [ (model-null)/(1-null) ]:  0.006461496
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq      log_fr
##          3.29836          -0.13117           0.19541           0.14232          -0.02270           0
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4369 Residual
## Null Deviance:       1928
## Residual Deviance: 1898  AIC: 1910
## log likelihood:  -948.8447
## Nagelkerke R2:  0.01920264
## % pres/err predicted correctly:  -470.7825
## % of predictable range [ (model-null)/(1-null) ]:  0.006710025
## ***************************
## model index:  21
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          0.94661           0.15612          -0.22684           1.91088           0.02613          -0
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4369 Residual
## Null Deviance:       1928
## Residual Deviance: 1898  AIC: 1910
## log likelihood:  -949.0542
## Nagelkerke R2:  0.01893564
## % pres/err predicted correctly:  -470.7198
## % of predictable range [ (model-null)/(1-null) ]:  0.00684216
## ***************************
## model index:  11
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq          I(pos^2)               pos  stimlen:log
##         3.252975         -0.126333          0.204079         -0.003431          0.159822         -0.0
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4369 Residual
## Null Deviance:       1928
```

```
## Residual Deviance: 1899  AIC: 1911
## log likelihood:  -949.4588
## Nagelkerke R2:  0.01842013
## % pres/err predicted correctly:  -470.8384
## % of predictable range [ (model-null)/(1-null) ]:  0.006592418
## ***************************
## model index:  17
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      3.5158      -0.1549       0.1315
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1905  AIC: 1911
## log likelihood:  -952.5719
## Nagelkerke R2:  0.01445004
## % pres/err predicted correctly:  -471.5453
## % of predictable range [ (model-null)/(1-null) ]:  0.005104156
## ***************************
## model index:  12
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)            stimlen           I(pos^2)               pos           log_freq  I(pos
##          3.257281          -0.129038          -0.005294           0.178660          -0.060411
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4368 Residual
## Null Deviance:       1928
## Residual Deviance: 1898  AIC: 1912
## log likelihood:  -948.7807
## Nagelkerke R2:  0.0192842
## % pres/err predicted correctly:  -470.8065
## % of predictable range [ (model-null)/(1-null) ]:  0.006659635
## ***************************
## model index:  18
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     2.97185     -0.08989      0.32675     -0.02268
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1904  AIC: 1912
## log likelihood:  -952.0387
## Nagelkerke R2:  0.01513042
```

```
## % pres/err predicted correctly:  -471.3912
## % of predictable range [ (model-null)/(1-null) ]:  0.005428453
## **************************
## model index:  10
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)            stimlen           log_freq           I(pos^2)               pos   stim
##          3.240476          -0.129364           0.089894          -0.004997           0.178975
##      log_freq:pos
##          0.077856
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4367 Residual
## Null Deviance:       1928
## Residual Deviance: 1897  AIC: 1913
## log likelihood:  -948.3831
## Nagelkerke R2:  0.01979069
## % pres/err predicted correctly:  -470.7371
## % of predictable range [ (model-null)/(1-null) ]:  0.006805716
## **************************
## model index:  20
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##    3.471371    -0.153614    -0.002887     0.155073
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1905  AIC: 1913
## log likelihood:  -952.5511
## Nagelkerke R2:  0.01447656
## % pres/err predicted correctly:  -471.5334
## % of predictable range [ (model-null)/(1-null) ]:  0.005129154
## **************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos     log_freq
##      2.4421       0.1012       0.1132
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1908  AIC: 1914
## log likelihood:  -954.2105
## Nagelkerke R2:  0.01235808
## % pres/err predicted correctly:  -471.805
```

```
## % of predictable range [ (model-null)/(1-null) ]:  0.004557431
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)           pos      log_freq  pos:log_freq
##      2.42758       0.10687       0.07774       0.01029
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1908  AIC: 1916
## log likelihood:  -954.0193
## Nagelkerke R2:  0.01260226
## % pres/err predicted correctly:  -471.7842
## % of predictable range [ (model-null)/(1-null) ]:  0.004601062
## ***************************
## model index:  9
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)           I(pos^2)                pos           log_freq  I(pos^2):log_freq       l
##          2.293383          -0.012395           0.203547          -0.023394          -0.008665
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4369 Residual
## Null Deviance:       1928
## Residual Deviance: 1906  AIC: 1918
## log likelihood:  -953.1598
## Nagelkerke R2:  0.01369971
## % pres/err predicted correctly:  -471.5907
## % of predictable range [ (model-null)/(1-null) ]:  0.005008516
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     log_freq
##     3.34836     -0.06959      0.08377
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1916  AIC: 1922
## log likelihood:  -958.0216
## Nagelkerke R2:  0.007486568
## % pres/err predicted correctly:  -472.7028
## % of predictable range [ (model-null)/(1-null) ]:  0.002667034
## ***************************
## model index:  16
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     2.46532      0.09017
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1919  AIC: 1923
## log likelihood:  -959.6872
## Nagelkerke R2:  0.005354825
## % pres/err predicted correctly:  -473.056
## % of predictable range [ (model-null)/(1-null) ]:  0.001923393
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq  stimlen:log_freq
##          3.33425          -0.06930           0.20386          -0.01514
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1916  AIC: 1924
## log likelihood:  -957.7643
## Nagelkerke R2:  0.00781572
## % pres/err predicted correctly:  -472.6518
## % of predictable range [ (model-null)/(1-null) ]:  0.002774408
## ***************************
## model index:  19
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.32851     -0.01111      0.18184
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1919  AIC: 1925
## log likelihood:  -959.3746
## Nagelkerke R2:  0.00575507
## % pres/err predicted correctly:  -472.9335
## % of predictable range [ (model-null)/(1-null) ]:  0.002181481
## ***************************
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
## (Intercept)      stimlen
##      3.5430      -0.0963
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1922  AIC: 1926
## log likelihood:  -960.8212
## Nagelkerke R2:  0.00390256
## % pres/err predicted correctly:  -473.3422
## % of predictable range [ (model-null)/(1-null) ]:  0.001320861
## **************************
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.795
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4374 Residual
## Null Deviance:       1928
## Residual Deviance: 1928  AIC: 1930
## log likelihood:  -963.8655
## Nagelkerke R2:  -6.230811e-16
## % pres/err predicted correctly:  -473.9696
## % of predictable range [ (model-null)/(1-null) ]:  0
## **************************
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                      AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                       by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)
kable(FLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	log_freq	pos	I(pos^2)	log_freq:pos	log_freq:I(pos^2)	stimlen:pos	stimlen:I(pos^2)
preserved ~ stimlen + pos + log_freq	1907.476	0.000000	1.000000	0.2683618	0.0062620	0.0844575	0.1318829	NA	NA	NA	NA	NA	NA	NA
					0.1281843									

Model	AIC	DeltaAIC	AICcWt	AggR2	(Intercept)	stimlen	log_freq	stimlen:log_freq	pos	log_freq:pos	I(pos^2)	stimlen:I(pos^2)	log_freq:I(pos^2)	stimlen:log_freq:I(pos^2)
preserved ~ stimlen + pos * log_freq	1908.7	0	0.2	0.1311154	0.0350857	0.1399307	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * log_freq + pos	1908.9	0.2	0.2	0.1278589	0.0149081	0.2026338	0.1318228	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos + log_freq	1909.4	0.7	0.1	0.1268451	0.0844907	0.1567351	NA	-0.0030434	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * log_freq + pos * log_freq	1909.6	0.9	0.1	0.1311738	0.0227029	0.1954131	0.1423522	0.0197597	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	1910.2	1.5	0.1	NA	NA	1.9108750	NA	-0.2268413	NA	NA	-0.2097085	0.0261339	NA	NA
preserved ~ stimlen * log_freq + I(pos^2) + pos	1910.3	1.6	0.1	0.1263332	0.0150851	0.2040786	0.1598220	NA	-0.0034307	NA	NA	NA	NA	NA
preserved ~ stimlen + pos	1911.3	2.6	0.1	0.1548783	NA	NA	0.1314884	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	1911.5	2.8	0.1	0.1290378	NA	0.1786397	NA	-0.0052994	-0.0079616	NA	NA	NA	NA	NA
preserved ~ stimlen * pos	1912.4	3.7	0.1	0.0898874	NA	NA	0.3267481	NA	NA	NA	NA	-0.0226805	NA	NA
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	1912.5	3.8	0.1	0.1293643	0.0199601	0.1789750	0.0778561	-0.0049967	NA	-0.0071602	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos	1913.1	4.4	0.1	0.1536145	NA	NA	0.1557730	NA	-0.0028866	NA	NA	NA	NA	NA
preserved ~ pos + log_freq	1914.6	5.9	0	0.1131574	0.0113574	0.1012441	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ pos * log_freq	1916.8	8.1	0	0.0777864	0.0777864	0.1068698	NA	NA	NA	NA	NA	NA	NA	NA

Model	AIC	DeltaAIC	AICcWt	NagR2	(Intercept)	stimlen	log_freq	stimlen:log_freq	pos	log_pos	log_freq:log_pos	I(pos^2)	log_freq:I(pos^2)	stimlen:I(pos^2)
preserved ~ (I(pos^2) + pos) * log_freq	1918.3	0	0.4	0.2	3.3	-0.1	-0.1	0.0233942	0.2	0.1	0.1	-0.1	-0.1	NA
preserved ~ stimlen + log_freq	1922.0	3.7	0.1	0.1	3.3	-0.1	-0.1	0.0695874	NA	NA	NA	NA	NA	NA
preserved ~ pos	1923.3	5.0	0.0	0.0	3.2	NA	NA	NA	0.1	0.1	0.1	NA	NA	NA
preserved ~ stimlen * log_freq	1923.5	5.2	0.0	0.0	3.3	-0.1	-0.1	0.0693040	NA	NA	NA	NA	NA	NA
preserved ~ I(pos^2) + pos	1924.1	5.8	0.0	0.0	3.2	NA	NA	NA	0.1	0.1	0.1	-0.1	NA	NA
preserved ~ stimlen	1925.1	6.8	0.0	0.0	3.2	-0.1	NA	0.0963028	NA	NA	NA	NA	NA	NA
preserved ~ 1	1929.2	10.9	0.0	0.0	3.2	NA	NA	NA	NA	NA	NA	NA	NA	NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen + pos + log_freq"
```

```
print(BestFLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos     log_freq
##     3.32006     -0.12818      0.13188      0.08446
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1899  AIC: 1907
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq]<-"hf"
PosDat$freq_bin[PosDat$log_freq <  median_freq]<-"lf"
```

```
PosDat$FLPFitted<-fitted(BestFLPModel)

HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]

HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitte
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
LF_Plot <- plot_len_pos_obs_predicted(LFDat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted
```

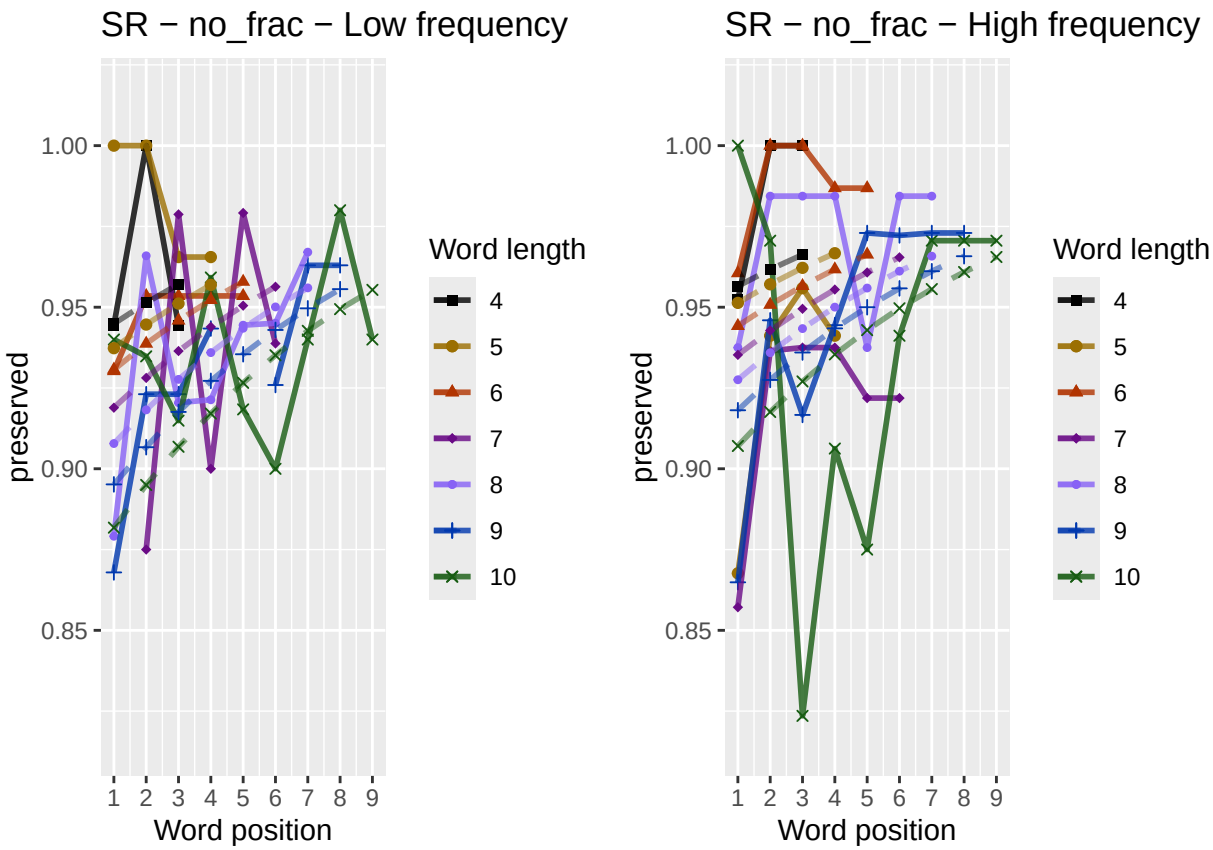
```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)
```




```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## **************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.206       -1.117
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1736  AIC: 1740
## log likelihood:  -868.2127
## Nagelkerke R2:  0.1200587
## % pres/err predicted correctly:  -428.7342
## % of predictable range [ (model-null)/(1-null) ]:  0.09523847
## **************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.0727       0.3577
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1834  AIC: 1838
## log likelihood:  -916.9993
## Nagelkerke R2:  0.05948015
## % pres/err predicted correctly:  -464.1224
## % of predictable range [ (model-null)/(1-null) ]:  0.02073227
## **************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     2.46532      0.09017
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1919  AIC: 1923
## log likelihood:  -959.6872
## Nagelkerke R2:  0.005354825
## % pres/err predicted correctly:  -473.056
## % of predictable range [ (model-null)/(1-null) ]:  0.001923393
## **************************
## model index:  3
##
```

```
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.32851     -0.01111      0.18184
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1919  AIC: 1925
## log likelihood:  -959.3746
## Nagelkerke R2:  0.00575507
## % pres/err predicted correctly:  -472.9335
## % of predictable range [ (model-null)/(1-null) ]:  0.002181481
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.5430      -0.0963
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1922  AIC: 1926
## log likelihood:  -960.8212
## Nagelkerke R2:  0.00390256
## % pres/err predicted correctly:  -473.3422
## % of predictable range [ (model-null)/(1-null) ]:  0.001320861
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.795
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4374 Residual
## Null Deviance:       1928
## Residual Deviance: 1928  AIC: 1930
## log likelihood:  -963.8655
## Nagelkerke R2:  -6.230811e-16
## % pres/err predicted correctly:  -473.9696
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
```

```
                        AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                          by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                              CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1740.425	0.00000	1	1	0.1200583	7.205870	NA	-1.116573	NA	NA	NA
preserved ~ CumPres	1837.999	97.57329	0	0	0.0594802	2.072728	0.3576607	NA	NA	NA	NA
preserved ~ pos	1923.374	182.94902	0	0	0.0053548	8.465322	NA	NA	NA	0.0901687	NA
preserved ~ (I(pos^2) + pos)	1924.749	184.32376	0	0	0.0057552	1.328512	NA	NA	-0.0111058	0.1818352	NA
preserved ~ stimlen	1925.642	185.21698	0	0	0.0039026	6.542966	NA	NA	NA	NA	-0.0963028
preserved ~ 1	1929.731	189.30569	0	0	0.0000000	2.794907	NA	NA	NA	NA	NA

```
if(DoSimulations){
      BestMEModelFormulaRnd <- BestMEModelFormula
    if(grepl("CumPres",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
    }else if(grepl("CumErr",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
    }

    RndModelAIC<-numeric(length=RandomSamples)
    for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
      PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
      PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
      BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                          family="binomial",data=PosDat)
      RndModelAIC[rindex] <- BestModelRnd$aic
    }
    ModelNames<-c(paste0("***",BestMEModelFormula),
                  rep(BestMEModelFormulaRnd,RandomSamples))
    AICValues <- c(BestMEModel$aic,RndModelAIC)
    BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
    BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
                              data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
```

```
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
                              data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

    write.csv(BestMEModelRndDF,
              paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                     "_best_main_effects_model_with_random_cum_term.csv"),
              row.names = FALSE)
}
```

```
syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                        RunLabel,"_",CurTask,
                                        "_syllable_component_summary.csv"),row.names = FALSE)
kable(syll_component_summary)
```

syll_component	MeanPres	N
1	0.9457093	571
O	0.9329389	2028
P	0.9722222	36
S	0.8932806	253
V	0.9616678	1487

```
#  main effects models for data without satellite positions

keep_components = c("O","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                              stimlen,stim,pos,
                              preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.281       -1.230
##
## Degrees of Freedom: 4085 Total (i.e. Null);  4084 Residual
## Null Deviance:       1736
## Residual Deviance: 1535  AIC: 1539
## log likelihood:  -767.2911
## Nagelkerke R2:  0.1391628
## % pres/err predicted correctly:  -375.5215
```

```
## % of predictable range [ (model-null)/(1-null) ]:  0.1108032
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.1065       0.4018
##
## Degrees of Freedom: 4085 Total (i.e. Null);  4084 Residual
## Null Deviance:       1736
## Residual Deviance: 1644  AIC: 1648
## log likelihood:  -821.7606
## Nagelkerke R2:  0.06487449
## % pres/err predicted correctly:  -412.9927
## % of predictable range [ (model-null)/(1-null) ]:  0.02231102
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.20512     -0.02322      0.28971
##
## Degrees of Freedom: 4085 Total (i.e. Null);  4083 Residual
## Null Deviance:       1736
## Residual Deviance: 1725  AIC: 1731
## log likelihood:  -862.5151
## Nagelkerke R2:  0.007981351
## % pres/err predicted correctly:  -421.1195
## % of predictable range [ (model-null)/(1-null) ]:  0.003118642
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     2.48930      0.09775
##
## Degrees of Freedom: 4085 Total (i.e. Null);  4084 Residual
## Null Deviance:       1736
## Residual Deviance: 1728  AIC: 1732
## log likelihood:  -863.791
## Nagelkerke R2:  0.00618189
## % pres/err predicted correctly:  -421.5014
## % of predictable range [ (model-null)/(1-null) ]:  0.00221675
## ***************************
## model index:  5
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##     3.47007     -0.08032
##
## Degrees of Freedom: 4085 Total (i.e. Null);  4084 Residual
## Null Deviance:       1736
## Residual Deviance: 1733  AIC: 1737
## log likelihood:  -866.2631
## Nagelkerke R2:  0.002691971
## % pres/err predicted correctly:  -422.0618
## % of predictable range [ (model-null)/(1-null) ]:  0.0008932503
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.847
##
## Degrees of Freedom: 4085 Total (i.e. Null);  4085 Residual
## Null Deviance:       1736
## Residual Deviance: 1736  AIC: 1738
## log likelihood:  -868.168
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -422.44
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",
                 CurTask,
                 "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1538.582	0.0000	1	1	0.139162	8.281284	NA	-1.23043	NA	NA	NA
preserved ~ CumPres	1647.521	108.9388	0	0	0.064874	3.106545	0.4017587	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1731.030	192.4480	0	0	0.007981	2.205122	NA	NA	-0.0232249	0.2897055	NA
preserved ~ pos	1731.582	192.9997	0	0	0.006181	2.489302	NA	NA	NA	0.0977550	NA
preserved ~ stimlen	1736.526	197.9440	0	0	0.002692	3.470074	NA	NA	NA	NA	-0.0803158
preserved ~ 1	1738.336	199.7537	0	0	0.000000	2.847294	NA	NA	NA	NA	NA

```r
#  main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
#  also reduces data)

keep_components = c("O","V")
OVData <- PosDat[PosDat$syll_component %in% keep_components,]
OVData <- OVData %>% select(stim_number,
                            stimlen,stim,pos,
                            preserved,syll_component)
OVData$CumPres <- CalcCumPres(OVData)
OVData$CumErr <- CalcCumErrFromPreserved(OVData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVData,MEModelEquations)
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.179       -1.225
##
## Degrees of Freedom: 3514 Total (i.e. Null);  3513 Residual
## Null Deviance:       1495
## Residual Deviance: 1383  AIC: 1387
## log likelihood:  -691.6203
## Nagelkerke R2:  0.09064329
## % pres/err predicted correctly:  -339.9499
## % of predictable range [ (model-null)/(1-null) ]:  0.06539319
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.1923       0.4137
##
## Degrees of Freedom: 3514 Total (i.e. Null);  3513 Residual
## Null Deviance:       1495
## Residual Deviance: 1429  AIC: 1433
## log likelihood:  -714.4819
## Nagelkerke R2:  0.05404542
## % pres/err predicted correctly:  -356.9255
## % of predictable range [ (model-null)/(1-null) ]:  0.0188599
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
```

```
## Coefficients:
## (Intercept)          pos
##      2.4323       0.1151
##
## Degrees of Freedom: 3514 Total (i.e. Null);  3513 Residual
## Null Deviance:       1495
## Residual Deviance: 1484  AIC: 1488
## log likelihood:  -742.229
## Nagelkerke R2:  0.008982495
## % pres/err predicted correctly:  -362.6375
## % of predictable range [ (model-null)/(1-null) ]:  0.003202183
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.18570     -0.02143      0.28955
##
## Degrees of Freedom: 3514 Total (i.e. Null);  3512 Residual
## Null Deviance:       1495
## Residual Deviance: 1483  AIC: 1489
## log likelihood:  -741.2761
## Nagelkerke R2:  0.01054186
## % pres/err predicted correctly:  -362.3239
## % of predictable range [ (model-null)/(1-null) ]:  0.004061824
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.846
##
## Degrees of Freedom: 3514 Total (i.e. Null);  3514 Residual
## Null Deviance:       1495
## Residual Deviance: 1495  AIC: 1497
## log likelihood:  -747.7079
## Nagelkerke R2:  3.203952e-16
## % pres/err predicted correctly:  -363.8057
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##     3.30272     -0.05926
```

```
##
## Degrees of Freedom: 3514 Total (i.e. Null);  3513 Residual
## Null Deviance:       1495
## Residual Deviance: 1494  AIC: 1498
## log likelihood:  -746.7948
## Nagelkerke R2:  0.001498984
## % pres/err predicted correctly:  -363.6263
## % of predictable range [ (model-null)/(1-null) ]:  0.0004917742
## ***************************
```

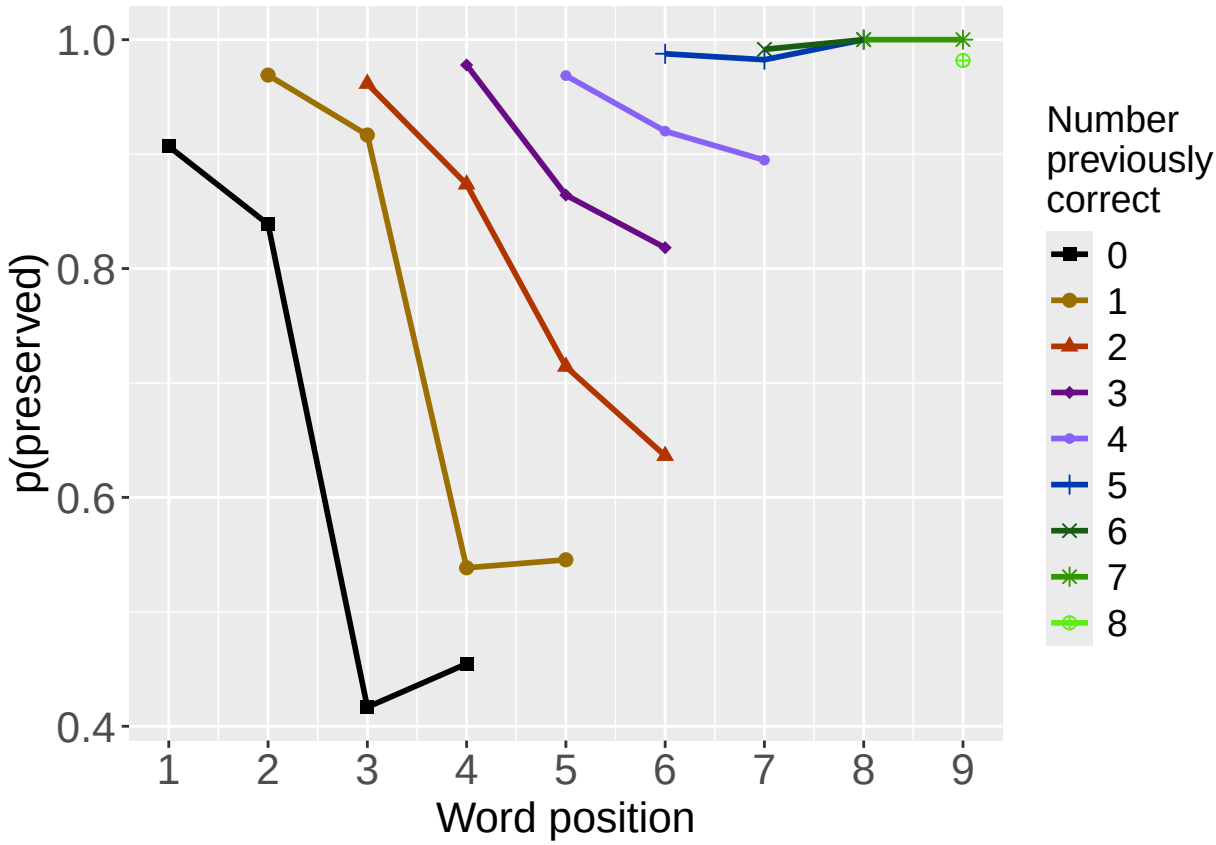
```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                 "_OV_main_effects_model_summary.csv"),row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1387.241	0.00000	1	1	0.0906433	3.179148	NA	-1.225267	NA	NA	NA
preserved ~ CumPres	1432.964	45.72328	0	0	0.0540452	4.192290	0.4137374	NA	NA	NA	NA
preserved ~ pos	1488.458	101.21735	0	0	0.0089823	3.432297	NA	NA	NA	0.1150602	NA
preserved ~ (I(pos^2) + pos)	1488.552	101.31159	0	0	0.0105419	2.185697	NA	NA	-0.0214286	0.2895478	NA
preserved ~ 1	1497.416	110.17515	0	0	0.0000000	2.845632	NA	NA	NA	NA	NA
preserved ~ stimlen	1497.590	110.34890	0	0	0.0014990	0.302716	NA	NA	NA	NA	-0.0592577

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

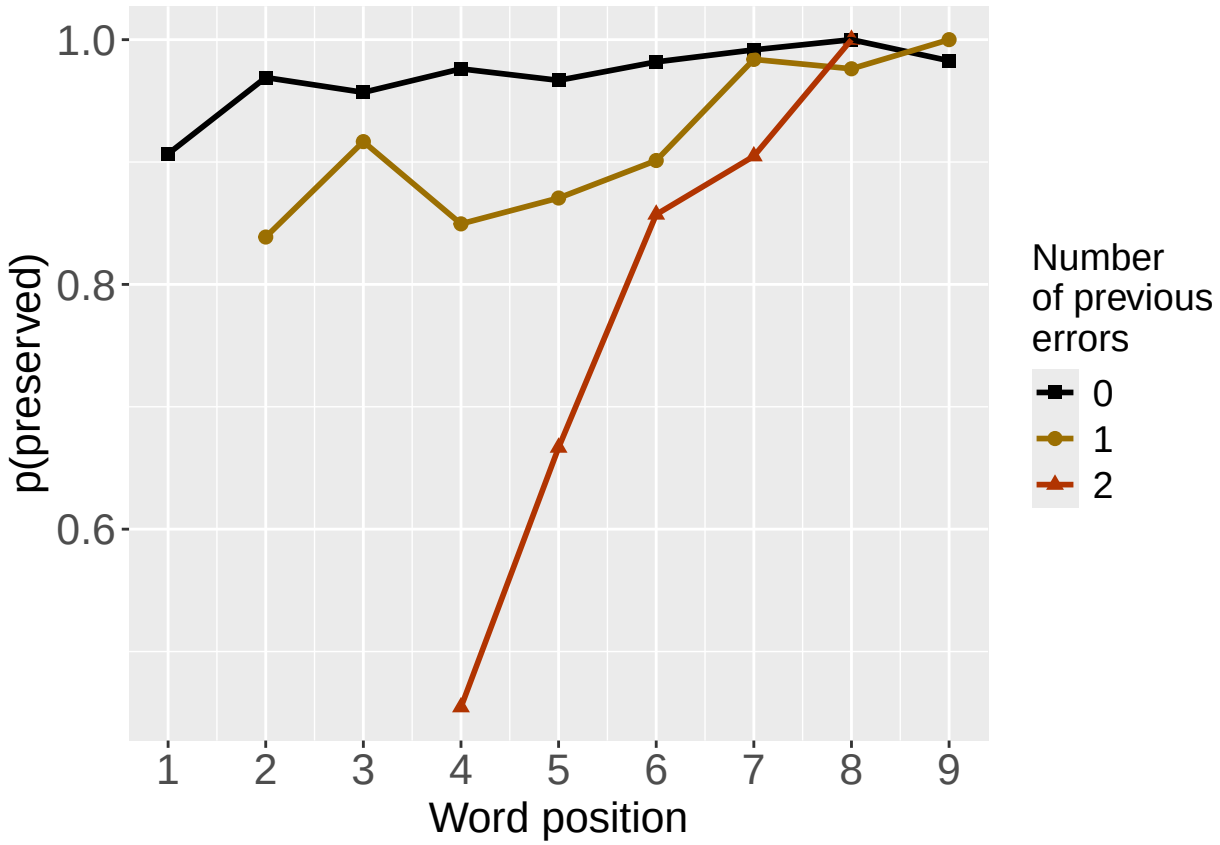
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

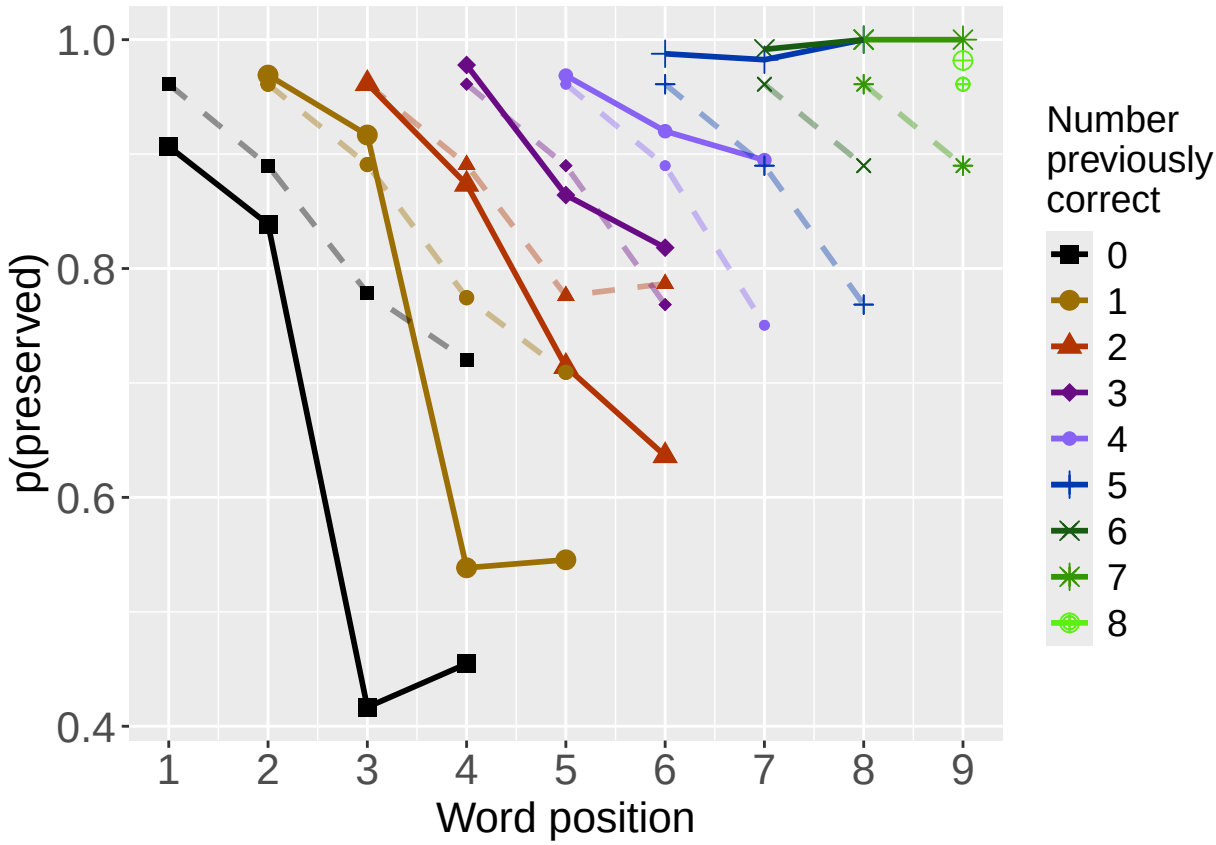
```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

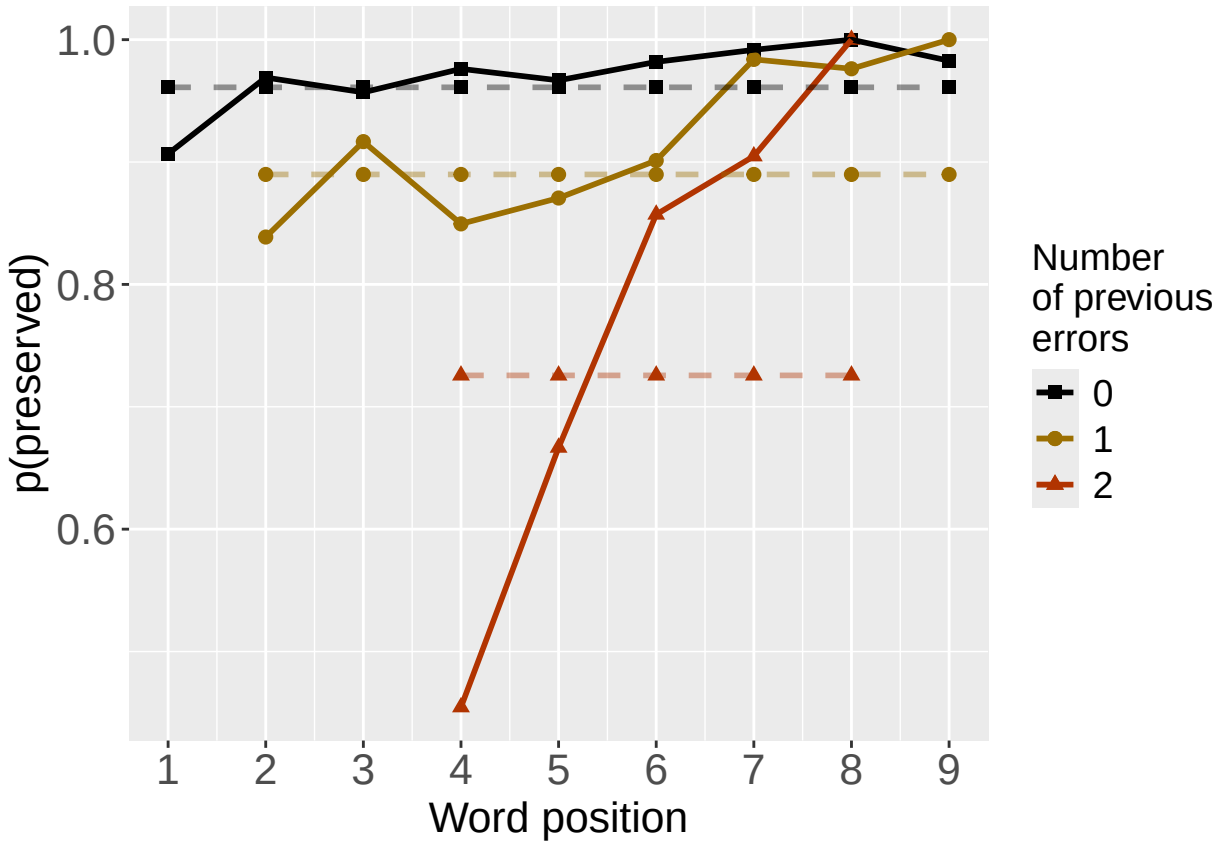
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```r
CumAICSummary <- NULL
```

```r
########
# level 2 -- Add position squared (quadratic with position)
########

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos
##    2.181649    -1.521630     0.001134     0.328231
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1660  AIC: 1668
## log likelihood:  -829.8892
## Nagelkerke R2:  0.1667066
## % pres/err predicted correctly:  -416.8886
## % of predictable range [ (model-null)/(1-null) ]:  0.1201783
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.206       -1.117
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1736  AIC: 1740
## log likelihood:  -868.2127
## Nagelkerke R2:  0.1200587
## % pres/err predicted correctly:  -428.7342
## % of predictable range [ (model-null)/(1-null) ]:  0.09523847
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.32851     -0.01111      0.18184
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1919  AIC: 1925
## log likelihood:  -959.3746
## Nagelkerke R2:  0.00575507
## % pres/err predicted correctly:  -472.9335
## % of predictable range [ (model-null)/(1-null) ]:  0.002181481
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	1667.778	0.00000	1	1	0.1667066	2.181649	-1.521630	0.0011342	0.3282315
preserved ~ CumErr	1740.425	72.64692	0	0	0.1200587	3.205870	-1.116573	NA	NA
preserved ~ I(pos^2) + pos	1924.749	256.97068	0	0	0.0057551	2.328512	NA	-0.0111058	0.1818352

```
########
# level 2 -- Add length
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ****************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.206       -1.117
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1736  AIC: 1740
## log likelihood:  -868.2127
## Nagelkerke R2:  0.1200587
## % pres/err predicted correctly:  -428.7342
## % of predictable range [ (model-null)/(1-null) ]:  0.09523847
## ****************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      stimlen
##     3.35119     -1.11027     -0.01902
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1736  AIC: 1742
## log likelihood:  -868.1097
## Nagelkerke R2:  0.1201851
## % pres/err predicted correctly:  -428.9392
## % of predictable range [ (model-null)/(1-null) ]:  0.09480688
## ****************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      3.5430      -0.0963
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1922  AIC: 1926
## log likelihood:  -960.8212
## Nagelkerke R2:  0.00390256
## % pres/err predicted correctly:  -473.3422
## % of predictable range [ (model-null)/(1-null) ]:  0.001320861
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr	1740.425	0.000000	1.0000000	0.7103397	0.1200587	3.205870	-1.116573	NA
preserved ~ CumErr + stimlen	1742.219	1.794069	0.4077772	0.2896603	0.1201851	3.351191	-1.110269	-0.0190209
preserved ~ stimlen	1925.642	185.216982	0.0000000	0.0000000	0.0039026	3.542966	NA	-0.0963028

```
########
# level 2 -- add cumulative preserved
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres
##       2.445       -1.154         0.380
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1644  AIC: 1650
## log likelihood:  -821.8093
## Nagelkerke R2:  0.1764377
## % pres/err predicted correctly:  -414.2012
## % of predictable range [ (model-null)/(1-null) ]:  0.1258362
## ***************************
## model index:  1
##
```

```
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.206       -1.117
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1736  AIC: 1740
## log likelihood:  -868.2127
## Nagelkerke R2:  0.1200587
## % pres/err predicted correctly:  -428.7342
## % of predictable range [ (model-null)/(1-null) ]:  0.09523847
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.0727       0.3577
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1834  AIC: 1838
## log likelihood:  -916.9993
## Nagelkerke R2:  0.05948015
## % pres/err predicted correctly:  -464.1224
## % of predictable range [ (model-null)/(1-null) ]:  0.02073227
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	1649.619	0.00000	1	1	0.1764377	2.444848	-1.154033	0.3799504
preserved ~ CumErr	1740.425	90.80677	0	0	0.1200587	3.205870	-1.116573	NA
preserved ~ CumPres	1837.999	188.38005	0	0	0.0594802	2.072728	NA	0.3576607

```
########
# level 2 -- Add linear position (NOT quadratic)
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## **************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr          pos
##      2.1690      -1.5214       0.3371
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1660  AIC: 1666
## log likelihood:  -829.8917
## Nagelkerke R2:  0.1667036
## % pres/err predicted correctly:  -416.9084
## % of predictable range [ (model-null)/(1-null) ]:  0.1201365
## **************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.206       -1.117
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1736  AIC: 1740
## log likelihood:  -868.2127
## Nagelkerke R2:  0.1200587
## % pres/err predicted correctly:  -428.7342
## % of predictable range [ (model-null)/(1-null) ]:  0.09523847
## **************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     2.46532      0.09017
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1919  AIC: 1923
## log likelihood:  -959.6872
## Nagelkerke R2:  0.005354825
## % pres/err predicted correctly:  -473.056
## % of predictable range [ (model-null)/(1-null) ]:  0.001923393
## **************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr + pos	1665.783	0.00000	1	1	0.1667036	2.169014	-1.521443	0.3371469
preserved ~ CumErr	1740.425	74.64204	0	0	0.1200587	3.205870	-1.116573	NA
preserved ~ pos	1923.374	257.59105	0	0	0.0053548	2.465322	NA	0.0901687

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_main_effects_plus_one_model_summary.csv"),row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + CumPres	1649.619	0.000000	1.000000	1.000000	0.1764377	2.444848	-1.154033	NA	NA	NA	0.3799504
preserved ~ CumErr + pos	1665.780	0.000000	1.000000	1.000000	0.1667036	2.169014	-1.521443	NA	0.3371469	NA	NA
preserved ~ CumErr + I(pos^2) + pos	1667.778	0.000000	1.000000	1.000000	0.1667036	2.181649	-1.521630	0.0011342	0.3282315	NA	NA
preserved ~ CumErr	1740.425	72.646920	0.000000	0.000000	0.1200587	3.205870	-1.116573	NA	NA	NA	NA
preserved ~ CumErr	1740.425	0.000000	1.000000	0.7103397	0.1200587	3.205870	-1.116573	NA	NA	NA	NA
preserved ~ CumErr	1740.429	0.806767	0.000000	0.000000	0.1200587	3.205870	-1.116573	NA	NA	NA	NA
preserved ~ CumErr	1740.427	4.642039	0.000000	0.000000	0.1200587	3.205870	-1.116573	NA	NA	NA	NA
preserved ~ CumErr + stimlen	1742.219	1.794069	0.407777	0.289660	0.1201853	3.351191	-1.110269	NA	NA	0.0190209	NA
preserved ~ CumPres	1837.999	188.380054	0.000000	0.000000	0.0059480	2.072728	NA	NA	NA	NA	0.3576607
preserved ~ pos	1923.374	257.591055	0.000000	0.000000	0.0053548	2.465322	NA	NA	0.0901687	NA	NA
preserved ~ I(pos^2) + pos	1924.749	256.970679	0.000000	0.000000	0.0057552	2.328512	NA	-0.0111058	0.1818352	NA	NA
preserved ~ stimlen	1925.641	285.216982	0.000000	0.000000	0.0039026	2.542966	NA	NA	NA	0.0963028	NA

```
# explore influence of frequency and length

if(grepl("stimlen",BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq")
  )
}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
    Level3ModelEquations<-c(
```

```
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq"),
      paste0(BestModelFormulaL2," + stimlen"),
      paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)
```

```
## ****************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres      stimlen     log_freq
##     3.45826     -1.10663      0.42503     -0.14251      0.08355
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4370 Residual
## Null Deviance:       1928
## Residual Deviance: 1623  AIC: 1633
## log likelihood:  -811.6438
## Nagelkerke R2:  0.1886296
## % pres/err predicted correctly:  -413.8789
## % of predictable range [ (model-null)/(1-null) ]:  0.1265148
## ****************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres      stimlen
##      3.6597      -1.1139       0.4232      -0.1699
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1628  AIC: 1636
## log likelihood:  -814.1164
## Nagelkerke R2:  0.1856693
## % pres/err predicted correctly:  -413.8645
## % of predictable range [ (model-null)/(1-null) ]:  0.1265452
## ****************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres     log_freq
##      2.4350      -1.1352       0.3936       0.1167
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1633  AIC: 1641
## log likelihood:  -816.5763
## Nagelkerke R2:  0.1827209
## % pres/err predicted correctly:  -414.1744
## % of predictable range [ (model-null)/(1-null) ]:  0.1258926
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres
##       2.445       -1.154        0.380
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1644  AIC: 1650
## log likelihood:  -821.8093
## Nagelkerke R2:  0.1764377
## % pres/err predicted correctly:  -414.2012
## % of predictable range [ (model-null)/(1-null) ]:  0.1258362
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.795
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4374 Residual
## Null Deviance:       1928
## Residual Deviance: 1928  AIC: 1930
## log likelihood:  -963.8655
## Nagelkerke R2:  -6.230811e-16
## % pres/err predicted correctly:  -473.9696
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

AICSummary<-data.frame(Model=Level3Res$Model,
                       AIC=Level3Res$AIC,
```

```
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                          by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                            CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres	log_freq	stimlen
preserved ~ CumErr + CumPres + stimlen + log_freq	1633.288	0.0000001	1.000000	0.8005108	0.8188629	6.458264	-1.106630	0.4250320	0.0835486	-0.1425093
preserved ~ CumErr + CumPres + stimlen	1636.233	2.9452460	0.2293232	0.1835737	0.7185669	3.659688	-1.113926	0.4232218	NA	-0.1699395
preserved ~ CumErr + CumPres + log_freq	1641.153	7.8649680	0.0195949	0.0156860	0.0182720	2.435042	-1.135210	0.3936153	0.1166804	NA
preserved ~ CumErr + CumPres	1649.619	16.3309690	0.0002843	0.0002276	0.6176437	7.444848	-1.154033	0.3799504	NA	NA
preserved ~ 1	1929.732	296.4434290	0.0000000	0.0000000	0.0000000	2.794907	NA	NA	NA	NA

```
# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions
```

```
## Single term deletions
##
## Model:
## preserved ~ CumErr + CumPres + stimlen + log_freq
##          Df Deviance    AIC
## CumErr    1   1798.5 1806.5
## CumPres   1   1731.7 1739.7
## stimlen   1   1633.2 1641.2
## log_freq  1   1628.2 1636.2
## <none>        1623.3 1633.3
```

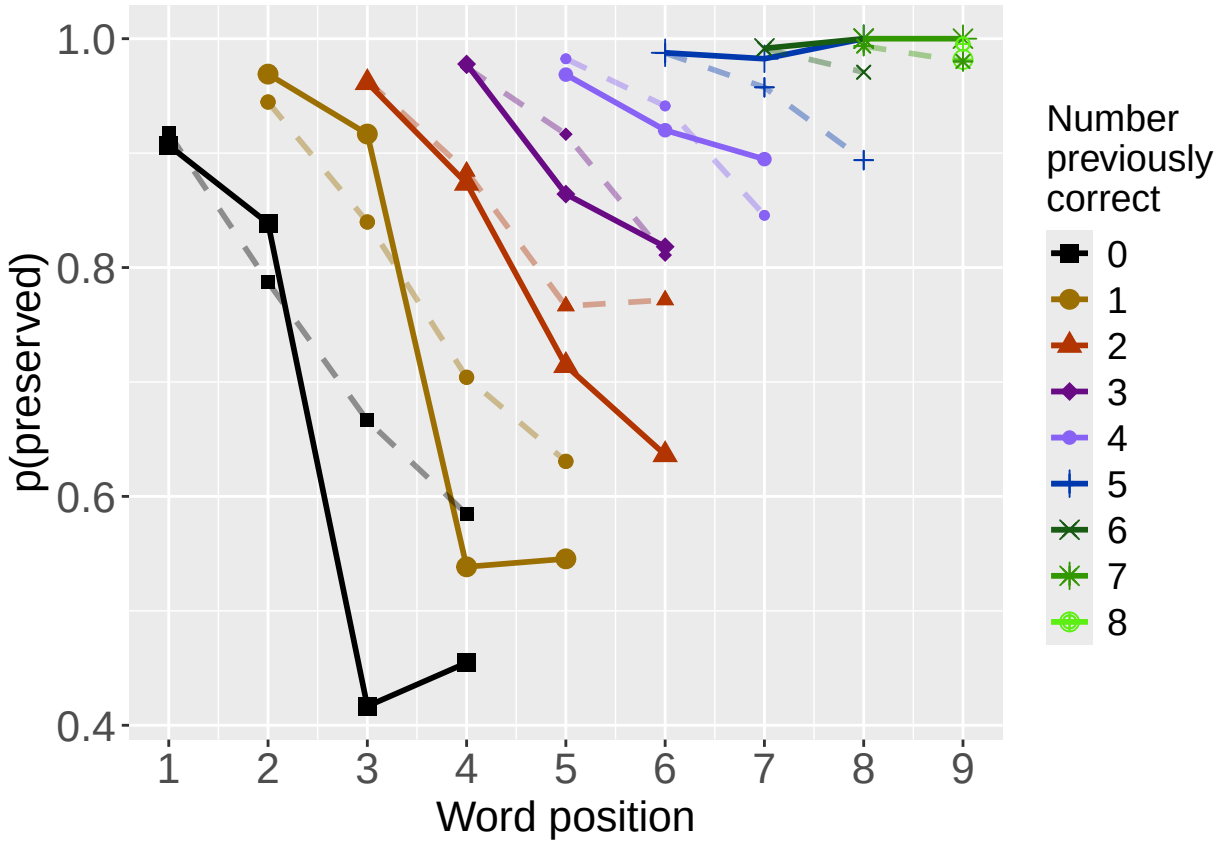
```
################################
# Single deletions from best model
################################
```

```
write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                    RunLabel,"_",CurTask,
                                    "_best_model_single_term_deletions.csv"),
          row.names = TRUE)
```

```
# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

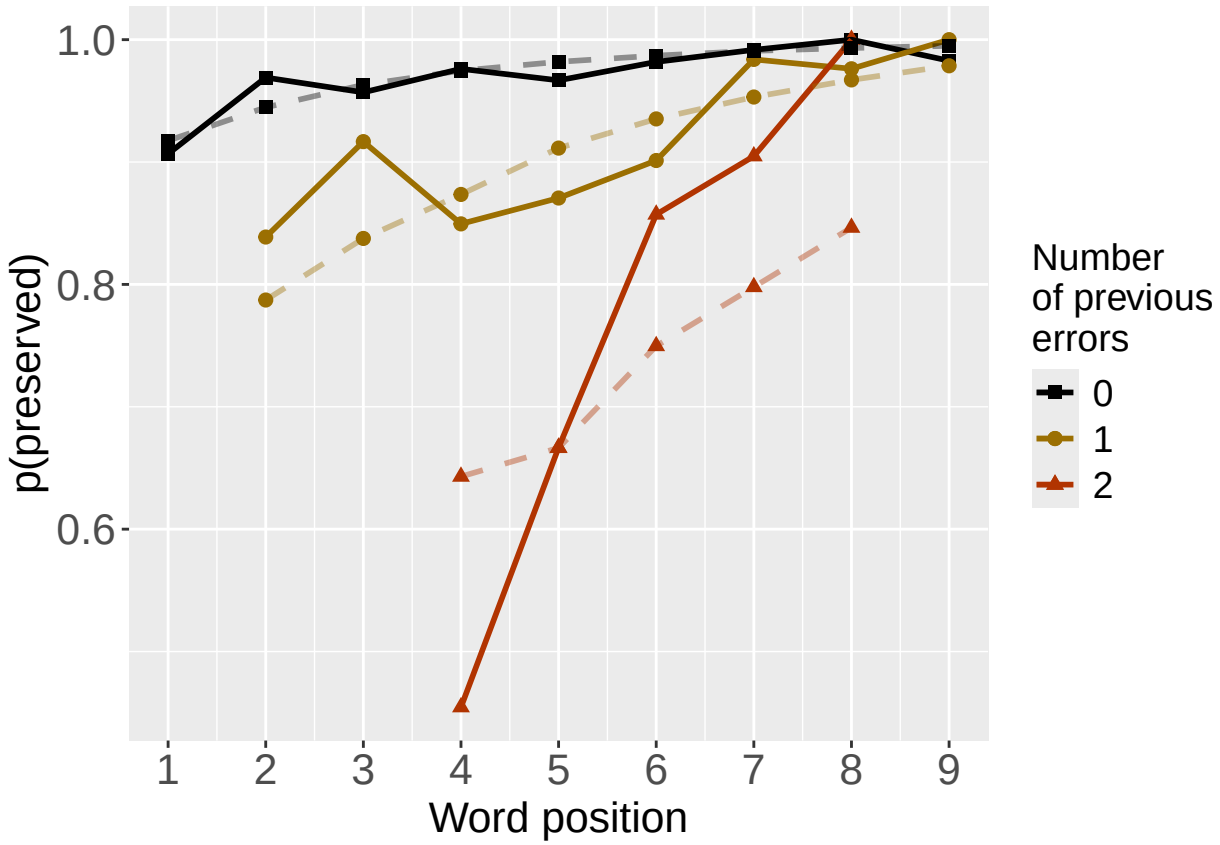
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",
                 RunLabel,"_",CurTask,
                 "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =
```

```
## Saving 6.5 x 4.5 in image
```

```
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "
```

```
## Saving 6.5 x 4.5 in image
```

```
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
  # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                        family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
```

```
  }
  ModelNames<-c(paste0("***",BestModelFormulaL3),
                rep(BestModelFormulaL3Rnd,RandomSamples))
  AICValues <- c(BestModelL3$aic,RndModelAIC)
  BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random average"),
                                     AIC=c(mean(RndModelAIC))))
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random SD"),
                                     AIC=c(sd(RndModelAIC))))
  write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                  "_best_model_with_random_cum_term.csv"),
            row.names = FALSE)
}
```

```
# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)
```

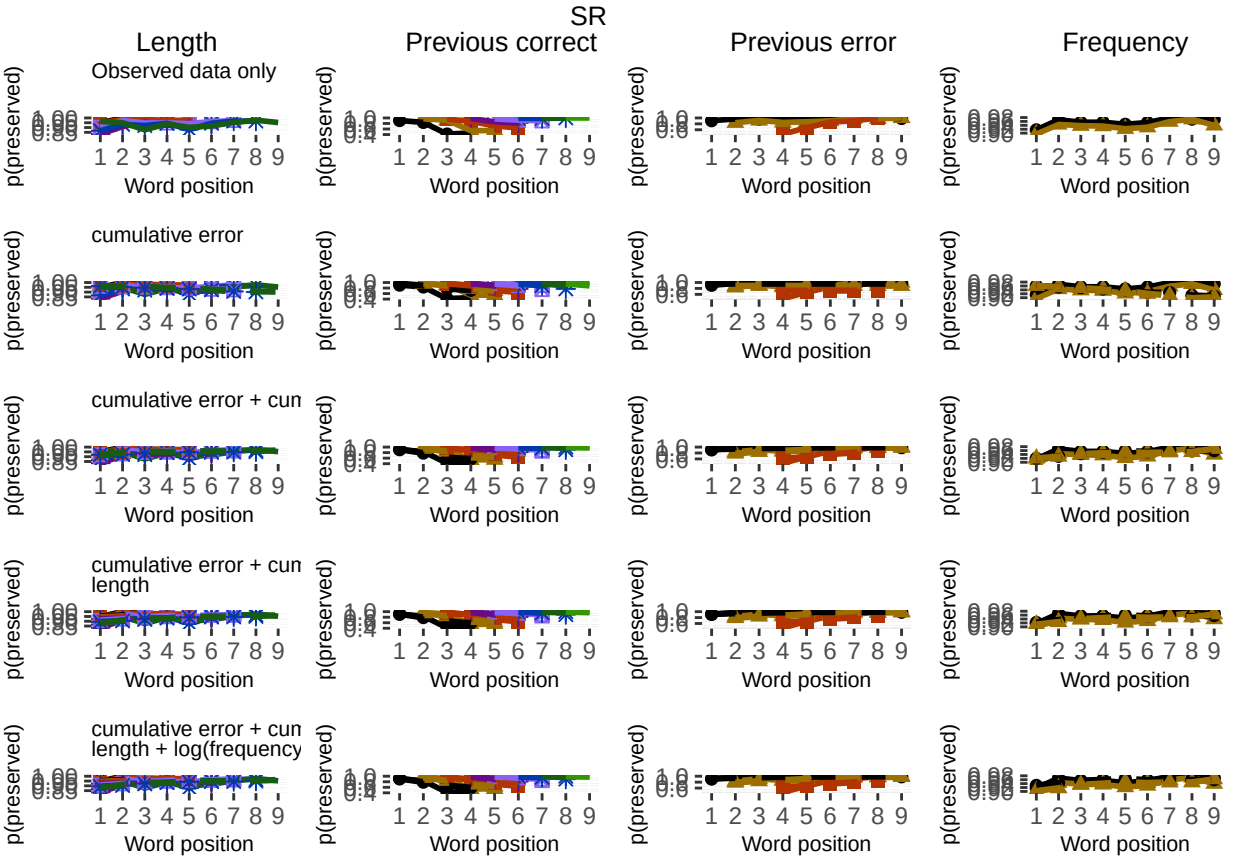
```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.206       -1.117
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4373 Residual
## Null Deviance:       1928
## Residual Deviance: 1736  AIC: 1740
## log likelihood:  -868.2127
## Nagelkerke R2:  0.1200587
## % pres/err predicted correctly:  -428.7342
## % of predictable range [ (model-null)/(1-null) ]:  0.09523847
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres
##       2.445       -1.154         0.380
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4372 Residual
## Null Deviance:       1928
## Residual Deviance: 1644  AIC: 1650
## log likelihood:  -821.8093
## Nagelkerke R2:  0.1764377
## % pres/err predicted correctly:  -414.2012
## % of predictable range [ (model-null)/(1-null) ]:  0.1258362
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres      stimlen
##      3.6597      -1.1139       0.4232      -0.1699
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4371 Residual
## Null Deviance:       1928
## Residual Deviance: 1628  AIC: 1636
## log likelihood:  -814.1164
## Nagelkerke R2:  0.1856693
## % pres/err predicted correctly:  -413.8645
## % of predictable range [ (model-null)/(1-null) ]:  0.1265452
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres      stimlen     log_freq
##     3.45826     -1.10663      0.42503     -0.14251      0.08355
##
## Degrees of Freedom: 4374 Total (i.e. Null);  4370 Residual
## Null Deviance:       1928
## Residual Deviance: 1623  AIC: 1633
## log likelihood:  -811.6438
## Nagelkerke R2:  0.1886296
## % pres/err predicted correctly:  -413.8789
## % of predictable range [ (model-null)/(1-null) ]:  0.1265148
## ***************************
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
```

```
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
ggsave(paste0(PlotName,".tif"),plot=FactorPlot,width = 360,height=400,units="mm",device="tiff",compressi
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```




```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                       CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	CumPres	stimlen	log_freq
McFadden	0.0955020	0.0536197	0.0050912	0.0037155

	CumErr	CumPres	stimlen	log_freq
SquaredCorrelation	0.0406481	0.0228231	0.0021616	0.0015882
Nagelkerke	0.0406481	0.0228231	0.0021616	0.0015882
Estrella	0.0440976	0.0247745	0.0023617	0.0017081

	deviance	deviance_explained
CumErr + CumPres + stimlen + log_freq	1623.288	304.4434
CumErr + CumPres + stimlen	1628.233	299.4982
CumErr + CumPres	1643.619	284.1125
CumErr	1736.425	191.3057
null	1927.731	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                                                 model deviance deviance_explained percent_explained
## CumErr + CumPres + stimlen + log_freq CumErr + CumPres + stimlen + log_freq 1623.288           304.4434         15.792837
## CumErr + CumPres + stimlen                       CumErr + CumPres + stimlen 1628.233           299.4982         15.536305
## CumErr + CumPres                                       CumErr + CumPres 1643.619           284.1125         14.738179
## CumErr                                                       CumErr 1736.425           191.3057          9.923879
## null                                                           null 1927.731             0.0000          0.000000
##                                       percent_of_explained_deviance increment_in_explained
## CumErr + CumPres + stimlen + log_freq                     100.00000               1.624356
## CumErr + CumPres + stimlen                                 98.37564               5.053721
## CumErr + CumPres                                           93.32192              30.484076
## CumErr                                                     62.83785              62.837846
## null                                                             NA               0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + CumPres + stimlen + log_freq	15.792837	100.00000	1.624356
CumErr + CumPres + stimlen	15.536305	98.37564	5.053721
CumErr + CumPres	14.738179	93.32192	30.484076
CumErr	9.923879	62.83785	62.837846
null	0.000000	NA	0.000000

```
                                   "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents
```

```
##          Nagelkerke
## CumErr   0.60469298
## CumPres  0.33952335
## stimlen  0.03215692
## log_freq 0.02362675
```

```
sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```



```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_sse_results_table.csv"),row.names = TRUE)
sse_table
```

```
##                                       model p_accounted_for model_deviance diff_CumErr diff_CumErr+CumPres+stimlen+log_freq
## 1                           preserved ~ CumErr       0.8314981       1736.425   0.0000000                       -0.109445103
## 2 preserved ~ CumErr+CumPres+stimlen+log_freq       0.9409432       1623.288   0.1094451                        0.000000000
## 3          preserved ~ CumErr+CumPres+stimlen       0.9425327       1628.233   0.1110346                        0.001589468
```

model	p_accounted_for	model_deviance
preserved ~ CumErr	0.8314981	1736.425
preserved ~ CumErr+CumPres+stimlen+log_freq	0.9409432	1623.288
preserved ~ CumErr+CumPres+stimlen	0.9425327	1628.233
preserved ~ CumErr+CumPres	0.9456407	1643.619

model	diff_CumErr	diff_CumErr+CumPres+stimlen+log_freq	diff_CumErr+CumPres+stimlen	diff_CumErr+CumPres
preserved ~ CumErr	0.0000000	-0.1094451	-0.1110346	-0.1141426
preserved ~ CumErr+CumPres+stimlen+log_freq	0.1094451	0.0000000	-0.0015895	-0.0046974
preserved ~ CumErr+CumPres+stimlen	0.1110346	0.0015895	0.0000000	-0.0031080
preserved ~ CumErr+CumPres	0.1141426	0.0046974	0.0031080	0.0000000

```
## 4                  preserved ~ CumErr+CumPres       0.9456407       1643.619   0.1141426                    0.004697448
##   diff_CumErr+CumPres+stimlen diff_CumErr+CumPres
## 1                -0.111034571        -0.114142551
## 2                -0.001589468        -0.004697448
## 3                 0.000000000        -0.003107981
## 4                 0.003107981         0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```